ROBOTICS IMPLEMENTATION REPORT

Hospital Health Monitoring Robot

Complete Hardware Guide + Viva Q&A; Bank

HNDCSAI-261F/P . NIBM School of Computing & Engineering
KIC-HNDCSAI-261F-014 & 015

ESP32 . 4WD . L298N . LM2596 Buck Converter . MAX30100 . HC-SR04 . DHT22

TABLE OF CONTENTS

PART A - HOW WE BUILT IT

1. Original Design vs. Final Implementation 3
2. Every Change We Made and Exactly Why 4
3. Final Hardware Architecture 8
4. Power System - Complete Design 9
5. Wiring and Component Integration 11
6. Software Architecture 13

PART B - VIVA PREPARATION

7. WHY Questions - Hardware and Design Decisions 15
8. HOW Questions - Technical Deep Dives 20
9. What-If and Scenario Questions 23
10. Quick-Fire Q&A; Reference Table 25

PART A - HOW WE BUILT IT

CHAPTER 1

Original Design vs. Final Implementation

1.1 What the Original Guide Specified

The NIBM HospitalRobot_Guide specified a dual-controller, 2-wheel differential drive architecture. An Arduino Uno handled all sensor reading, motor control, and RFID identification, while a ZY ESP32E acted purely as a WiFi gateway. The two boards communicated over UART serial at 115,200 baud using a CSV text protocol.

Parameter	Original Specification
Controller	Arduino Uno (main) + ZY ESP32E (WiFi only)
Drive system	2WD differential drive: 2x DC gear motors + 1 ball caster (rear)
Patient ID	RC522 RFID reader via SPI - scans patient wristband UID
Vitals sensors	MAX30102 (HR + SpO2 I2C) + MLX90614 (IR temperature, I2C)
Navigation	HC-SR04 ultrasonic: obstacle < 25cm = reverse + turn (dynamic avoidance)
Power	LM2596 buck converter to 5V rail; AMS1117-3.3 for 3.3V devices
Alerts	Arduino evaluates thresholds, sends to ESP32E, Firebase, FCM
Comms	Arduino CSV string over UART to ESP32E parses then Firebase write

1.2 Final System at a Glance

Due to hardware constraints discovered during prototyping, pin conflicts, sensor incompatibilities, and reliability issues, the system was significantly evolved from the original guide.

Parameter	Final Implementation
Controller	ESP32 DevKit only - handles sensors, motors, Firebase, navigation
Drive system	4WD: 4x DC gear motors for better traction and stability
Patient ID	RFID removed. IR sensor all-black stop-point detection replaces it
Vitals sensors	MAX30100 (HR + SpO2) + DHT22 (temperature + humidity)
Navigation	5x IR array for line following - stop-and-wait obstacle handling
Power	LM2596 buck converter to 5V rail shared, ESP32 via 5V VIN pin
Alerts	ESP32 evaluates thresholds locally, drives LEDs + Firebase flag
Comms	ESP32 writes directly to Firebase - no serial bridge needed

CHAPTER 2

Every Change We Made and Exactly Why

This chapter is the most important section for your viva. Every deviation from the guide is documented with the precise engineering reason your lecturer will expect. Read every WHY out loud before your viva.

Change 1 — 2-Wheel Drive to 4-Wheel Drive

ORIGINAL GUIDE	2WD differential drive: 2x DC gear motors plus 1 ball caster at the rear.
WHAT WE DID	4WD: 4x DC gear motors, all four wheels driven simultaneously by the L298N dual H-bridge.
WHY WE CHANGED IT	The original 2WD chassis with ball caster was unstable on linoleum hospital floor tiles. The ball caster would skid sideways when motors started suddenly, causing the robot to deviate from the line track. 4WD distributes torque across all four wheels, giving uniform traction, preventing skidding, and making line following far more accurate. With 4WD, turning is also more predictable: both left wheels reverse while both right wheels forward, creating a clean pivot turn rather than a drift. The additional two motors add approximately 400mA to current draw, which our LM2596 (3A rated) handles comfortably.

Change 2 — RFID Patient Identification Removed

ORIGINAL GUIDE	RC522 RFID reader on Arduino SPI (MOSI=D11, MISO=D12, SCK=D13, SS=D10, RST=A0) - patient scans wristband to identify themselves.
WHAT WE DID	RFID completely removed. Patient identity determined by black tape cross-markers on the track. When all 5 IR sensors simultaneously read LOW (all-black), the robot knows it is at a patient station.
WHY WE CHANGED IT	The RC522 requires 5 SPI pins. In the single-ESP32 architecture, after allocating pins for DHT22 (1), MAX30100 I2C (2), 5x IR sensors (5), HC-SR04 (2), and L298N motor driver (6), there are insufficient SPI-capable ESP32 GPIO pins without creating conflicts. The RC522 is also strictly 3.3V on its data lines, but several assigned GPIOs share voltage domains with 5V motor driver circuitry. Rather than add level shifters and risk integration failures close to the demo deadline, we replaced RFID with optical stop-point detection: no extra pins, no voltage incompatibility, and equally deterministic.

Change 3 — Arduino Uno + ESP32E Dual Controller to ESP32 Only

ORIGINAL GUIDE	Arduino Uno reads all sensors and controls motors. ESP32E is a dedicated WiFi gateway receiving data over UART serial CSV protocol.
WHAT WE DID	Single ESP32 handles all sensors, motors, navigation, and Firebase communication directly. No serial bridge, no CSV parsing.
WHY WE CHANGED IT	The dual-controller architecture introduced a critical failure point: if even one byte of the UART CSV frame was corrupted, the ESP32E would parse wrong patient data silently with no error flag. Arduino (16MHz) and ESP32 (240MHz) operate on different clock domains, making baud rate drift a real risk over time. Eliminating the serial bridge removes this entire failure category. The ESP32 has 34 GPIO pins, hardware I2C, hardware SPI, two hardware UART ports, and dual cores - it is fully capable of replacing the Arduino entirely.

Change 4 — Dynamic Obstacle Avoidance to Stop-and-Wait

ORIGINAL GUIDE	Obstacle < 25cm: motors stop, reverse 500ms, turn right ~90 degrees, continue forward. Robot attempts to navigate around the obstacle.
WHAT WE DID	Obstacle < 15cm: stop immediately, log to Firebase, wait 3 seconds, resume. No reversal or turning.
WHY WE CHANGED IT	Dynamic avoidance caused the robot to lose the line track entirely. After turning 90 degrees, the 5-sensor IR array had no line under any sensor and the robot would spin or head in the wrong direction. In a hospital corridor, obstacles are typically people or trolleys that will move within seconds - a brief stop-and-wait is more appropriate behaviour. A robot reversing aggressively in a hospital ward is also a safety hazard. The stop-and-wait also logs the event to Firebase (/robotStatus/currentState = OBSTACLE), alerting nursing staff.

Change 5 — MLX90614 IR Temperature Sensor Replaced by DHT22

ORIGINAL GUIDE	MLX90614 contactless infrared thermometer: I2C address 0x5A, measures body temperature without contact from 5-15cm distance.
WHAT WE DID	DHT22 digital temperature and humidity sensor: single-wire protocol on GPIO16, measures ambient temperature and relative humidity.
WHY WE CHANGED IT	The MLX90614 uses I2C at address 0x5A. The MAX30100 uses I2C at address 0x57. Both share SDA/SCL. During testing, simultaneous reads caused bus contention - one sensor would hold the bus during a slow read, timing out the other. The MLX90614 also requires precise targeting distance (5-15cm) and emissivity calibration for accurate skin readings, which is impossible to maintain on a moving robot. The DHT22 resolves both issues: dedicated single-wire protocol on its own GPIO pin eliminates bus sharing entirely, and adds humidity as an additional clinically relevant metric.

Change 6 — SSL Errors Fixed with NTP Time Sync

ORIGINAL GUIDE	Guide suggests Firebase ESP32 library with legacy auth token. No mention of SSL or time synchronisation requirements.
WHAT WE DID	Firebase_ESP_Client with mandatory NTP time sync before Firebase.begin(). config.signer.test_mode = true for prototype operation.
WHY WE CHANGED IT	Firebase uses HTTPS exclusively. Every TLS connection validates the server certificate's notBefore date against the device clock. The ESP32 has no hardware RTC and starts at Unix timestamp 0 (1 January 1970) on cold boot - decades before any valid certificate. Firebase rejects the handshake with an SSL error. The fix is to call configTime() with pool.ntp.org and poll until time(nullptr) > 100,000 before calling Firebase.begin(). This was not documented in the original guide and was discovered through repeated SSL connection failures during integration testing.

Change 7 — Cloud Alert Logic Moved to ESP32 Edge Processing

ORIGINAL GUIDE	Alert evaluation handled by Firebase Cloud Functions. Cloud function checks thresholds and triggers FCM push notification.
WHAT WE DID	ESP32 evaluates HR, SpO2, and temperature thresholds locally every 5 seconds. Sets /alerts/active flag and drives physical LEDs without cloud dependency.
WHY WE CHANGED IT	Firebase Cloud Functions require the Blaze paid billing plan to make outbound calls for FCM push notifications. The free Spark plan blocks all egress traffic. Rather than upgrade billing for a student prototype, we moved decision logic to the ESP32. This is also architecturally superior: latency drops from approximately 2 seconds (ESP32 to Firebase to Cloud Function to FCM to phone) to under 500ms. It also works when Firebase is unreachable - the LED alert fires regardless of cloud connectivity.

Change 8 — No Physical Feedback to LED Alert System Added

ORIGINAL GUIDE	Guide mentions optional buzzer or LED. No specific implementation provided.
WHAT WE DID	Green LED = normal vitals. Red LED = abnormal vitals. ESP32 GPIO drives LEDs via 470 ohm current-limiting resistors.
WHY WE CHANGED IT	A physical LED provides an ambient glanceable indicator visible from across the ward without requiring the nurse to check a phone app. It also provides immediate confirmation during demos that the sensor system is working. The 470 ohm resistors are mandatory: a 3.3V GPIO driving an LED without a current-limiting resistor would draw approximately 33mA (3.3V / ~100 ohm LED resistance), exceeding the ESP32 GPIO maximum of 12mA per pin and permanently damaging it.

Change 9 — 3.3V Regulated Rail Simplified to 5V Shared Rail

ORIGINAL GUIDE	Guide: Battery to LM2596 to 5V to AMS1117-3.3 to 3.3V to ESP32E VIN.
WHAT WE DID	Battery to LM2596 to 5V rail to ESP32 5V VIN pin. The ESP32's onboard AMS1117 regulator steps 5V down to 3.3V internally.
WHY WE CHANGED IT	The ESP32 DevKit module has an onboard AMS1117-3.3 LDO regulator rated at 800mA - sufficient for the ESP32 itself. Supplying 5V to the DevKit's VIN pin lets this onboard regulator do its job as designed. Adding an external AMS1117 module would create double regulation: LM2596 to 5V to external AMS1117 to 3.3V to ESP32 onboard AMS1117. This is wasteful. However, motor noise on the shared 5V rail must be filtered by 10uF plus 100nF decoupling capacitors at the ESP32 VIN pin.

Change 10 — Line Following Added as Primary Navigation System

ORIGINAL GUIDE	Original guide specifies basic forward patrol with obstacle avoidance. Navigation is not structured - robot roams freely.
WHAT WE DID	5-sensor IR array is the primary navigation system. Robot follows a predefined black line track. All-black reading triggers patient stop-point logic.
WHY WE CHANGED IT	Without a defined path, the robot would roam randomly, making it impossible to demonstrate reliable patient-visiting behaviour. A fixed line track ensures the robot visits each patient position in a predictable sequence - critical for a hospital prototype. The 5-sensor array provides proportional steering correction: outer sensors detect larger deviations and trigger sharper turns, while the centre sensor maintains straight-line travel. This makes navigation smoother at higher speeds than a 2 or 3-sensor design.

CHAPTER 3

Final Hardware Architecture

3.1 Component List

Component	Role and Connection
ESP32 DevKit v1	Main controller: WiFi, sensors, motors, Firebase
MAX30100	Pulse oximeter: HR + SpO2 via I2C (SDA=GPIO21, SCL=GPIO22)
DHT22	Temperature + Humidity via single-wire on GPIO16
HC-SR04	Ultrasonic distance: TRIG=GPIO17, ECHO=GPIO23 (via voltage divider)
5x IR sensors	Line following array: GPIO32, 34, 35, 18, 19
L298N motor driver	Dual H-bridge: drives all 4 motors paired left and right
4x DC gear motors	Drive system: all four wheels driven for maximum traction
LM2596 buck converter	7.4V battery to 5V regulated output for all logic components
2x 18650 Li-ion	7.4V nominal, ~2500mAh: powers entire system
Red LED + 470 ohm	Alert indicator: abnormal vitals
Green LED + 470 ohm	Status indicator: normal vitals / system running
Power switch	Main on/off between battery and buck converter input

3.2 GPIO Pin Allocation

ESP32 GPIO	Connected To
GPIO 16	DHT22 data pin (single-wire protocol)
GPIO 17	HC-SR04 TRIG (output, 3.3V sufficient to trigger)
GPIO 21	I2C SDA: MAX30100
GPIO 22	I2C SCL: MAX30100
GPIO 23	HC-SR04 ECHO (input) via 1k/2k ohm voltage divider to limit to 3.3V
GPIO 32	IR Sensor S1 far left
GPIO 34	IR Sensor S2 centre-left
GPIO 35	IR Sensor S3 centre
GPIO 18	IR Sensor S4 centre-right
GPIO 19	IR Sensor S5 far right
GPIO 26	L298N ENA: left motor pair PWM speed

GPIO 27	L298N IN1: left motor direction A
GPIO 14	L298N IN2: left motor direction B
GPIO 25	L298N ENB: right motor pair PWM speed
GPIO 33	L298N IN3: right motor direction A
GPIO 13	L298N IN4: right motor direction B
GPIO 2	Green LED via 470 ohm to GND
GPIO 4	Red LED via 470 ohm to GND

CHAPTER 4

Power System - Complete Design

4.1 Why the Power System Matters

The power system is the most critical subsystem. A poorly designed power system causes random ESP32 resets (brownout), corrupted sensor readings, WiFi disconnections, and permanent component damage. Every decision here has a specific engineering reason.

4.2 Battery Specification

Parameter	Specification
Type	2x Samsung/LG 18650 Li-ion cells in SERIES
Voltage	7.4V nominal, 8.4V fully charged, 6.0V cutoff
Capacity	2500-3000mAh per cell
Peak discharge	Minimum 2C rating = 5,000mA peak - covers our 1.6A design load
Protection	Built-in PCB protection: overcharge, over-discharge, short-circuit

4.3 Current Budget

Consumer	Current Draw
ESP32 WiFi active transmitting	~170mA average (350mA peak burst)
MAX30100 LEDs active	~25mA
DHT22	~1.5mA
HC-SR04 pulsing	~15mA average
5x IR sensors	~50mA total (10mA each)
L298N logic quiescent	~36mA
4x DC motors running no load	~800mA (200mA each)
4x DC motors loaded	~1,600mA (400mA each)
Green and Red LEDs	~14mA total
TOTAL motors running WiFi on	~1,111mA = 1.1A average
Design current 30% margin	1.1A x 1.3 = 1.43A
Runtime at 2500mAh	2500 / 1100 = ~2.3 hours (exceeds 2hr requirement)

4.4 Buck Converter - Why It Is Needed

A resistor drops voltage by wasting energy as heat: P = I squared x R. At 1.1A load, dropping 3V (8V to 5V) wastes 3.3W continuously as heat, draining the battery rapidly. The LM2596 buck converter does the

same job at 75-80% efficiency using rapid transistor switching and an inductor - almost no heat, far longer runtime. At 1.1A x 3V drop, a resistor wastes 3.3W. The LM2596 wastes only ~0.7W for the same conversion. Over 2 hours, that is 5.28Wh saved - directly translating to extra runtime.

4.5 Power Distribution Architecture

Point	Connection and Reasoning
Battery (+)	Main power switch then splits into TWO independent rails
Rail A motor power	Battery (+) to fuse (3A) to L298N VS terminal. RAW voltage, no regulation.
Rail B logic power	Battery (+) to LM2596 input to 5V output
5V rail feeds	ESP32 VIN pin, L298N VSS, IR sensors, HC-SR04, DHT22
ESP32 internal	Onboard AMS1117-3.3 steps 5V to 3.3V for internal logic and GPIO
MAX30100	ESP32 3.3V pin (onboard AMS1117 output, max 200mA - sufficient)
Common GND	Battery (-) = LM2596 GND = L298N GND = ESP32 GND = ALL sensor GNDs

WARNING: ALL ground connections MUST physically connect to a single common point. A disconnected ground causes I2C failure, wrong sensor readings, and serial garbage. Use thick wire (20AWG) for motor ground returns.

4.6 Decoupling Capacitors - Motor Noise Filtering

Component	Purpose and Placement
100nF ceramic capacitor	Across VCC-GND of every sensor and IC. Filters high-frequency motor switching noise. Mount as close to the chip as possible.
10uF electrolytic capacitor	Across ESP32 5V VIN pin and GND. Absorbs voltage sags during WiFi transmission spikes.
100uF electrolytic capacitor	Across LM2596 output terminals. Smooths output and handles load transients when all 4 motors start simultaneously.
1N4007 diodes	Across each motor terminal (cathode to +, anode to -). Catches back-EMF spikes when motors brake.

CHAPTER 5

Wiring and Component Integration

5.1 HC-SR04 Ultrasonic - Voltage Divider on ECHO Pin

> NOTE: The HC-SR04 operates at 5V and its ECHO pin outputs a 5V signal. The ESP32 GPIO pins are 3.3V tolerant only. Connecting 5V directly to an ESP32 GPIO will damage it. A resistor voltage divider is mandatory.

Divider calculation: ECHO connects to 1k ohm resistor, then to ESP32 GPIO23, then 2k ohm to GND. When ECHO is HIGH (5V): V at GPIO = 5 x (2,000 / (1,000 + 2,000)) = 5 x 0.667 = 3.33V. This is within the ESP32's 3.3V tolerance. The TRIG pin is an output from the ESP32 at 3.3V - sufficient to trigger the HC-SR04 (threshold approximately 2V). No divider needed on TRIG.

5.2 MAX30100 I2C - Pull-up Resistors

> NOTE: The MAX30100 is a 3.3V I2C device. The I2C bus requires pull-up resistors on both SDA and SCL lines to pull them HIGH when no device is driving them LOW. Without pull-ups, I2C will not function at all.

Connect 4.7k ohm resistors: SDA line to 4.7k ohm to 3.3V, and SCL line to 4.7k ohm to 3.3V. The resistors must pull to 3.3V (not 5V) because the MAX30100's I2C pins are 3.3V devices. Pulling to 5V would exceed the MAX30100's absolute maximum input voltage and damage it. Only ONE set of pull-up resistors per bus line is needed regardless of how many I2C devices share the bus.

5.3 LED Wiring - Current Limiting Resistors

> NOTE: Current limiting resistors are MANDATORY between every GPIO pin and LED. Without them, the LED draws 20-30mA which permanently damages the ESP32 GPIO pin (maximum 12mA per pin).

Calculation: ESP32 GPIO HIGH = 3.3V. LED forward voltage (red/green) approximately 2.0V. Desired current = 10mA. R = (3.3 - 2.0) / 0.010 = 130 ohms minimum. We use 470 ohms for safety margin: (3.3 - 2.0) / 470 = 2.8mA. Visible brightness and completely safe for the GPIO. For brighter LED use 220 ohms for 5.9mA - still within the 12mA limit.

5.4 L298N Motor Driver - 4WD Wiring

The L298N has two independent H-bridge channels (A and B). For 4WD, we parallel the motors: Left motor pair (front-left plus rear-left) connected to OUT1/OUT2 in parallel; Right motor pair connected to OUT3/OUT4 in parallel. Both motors on each side receive identical PWM signals and behave as one unit. The combined stall current of 2 parallel motors is approximately 800mA per side, within the L298N's 2A per channel rating.

L298N Pin	Connection
ENA GPIO26	PWM speed control: left motor pair (analogWrite 0-255)
IN1 GPIO27	Left pair direction A: HIGH=forward with IN2 LOW
IN2 GPIO14	Left pair direction B: opposite to IN1

ENB GPIO25	PWM speed control: right motor pair
IN3 GPIO33	Right pair direction A
IN4 GPIO13	Right pair direction B
OUT1+OUT2	Left motor pair (front-left and rear-left in parallel)
OUT3+OUT4	Right motor pair (front-right and rear-right in parallel)
VS	Battery positive (7.4-8.4V): raw battery, NOT through buck converter
VSS	5V from LM2596: logic supply for H-bridge driver circuits

WARNING: The L298N VS pin must connect to the battery directly, NOT through the buck converter. Motors need full battery voltage. Running motors from 5V buck output would starve them of voltage and overload the converter simultaneously.

CHAPTER 6

Software Architecture

6.1 Main Loop Priority Order

The ESP32 runs a priority-ordered main loop. Safety checks execute first, navigation last. This order is intentional.

Priority	Action
Priority 1 - WiFi watchdog	Check WiFi.status(). If disconnected, reconnect and return.
Priority 2 - Firebase send	Every 5s: read sensors, evaluate alerts, write to Firebase.
Priority 3 - Command poll	Every 2s: read /commands/driveCommand, execute motor action.
Priority 4 - Obstacle check	Every loop: HC-SR04 distance. If < 15cm, stop and wait 3s.
Priority 5 - Patient stop point	Every loop: check all IR sensors. If all LOW, stop 5s for measurement.
Priority 6 - Line following	Every loop (10ms delay): navigate based on IR sensor pattern.

6.2 MAX30100 on FreeRTOS Core 0

The MAX30100 requires continuous pox.update() calls. Missing this for even 50ms causes the beat detection to lose its tracking and return zero. Firebase HTTPS writes can block the main loop for 200-800ms during SSL handshake. To solve this, poxTask is pinned to Core 0 using xTaskCreatePinnedToCore(). The main loop runs on Core 1. Shared variables g_hr and g_spo2 are protected with portENTER_CRITICAL / portEXIT_CRITICAL and a portMUX_TYPE mutex to prevent race conditions between cores.

6.3 Alert Thresholds

Parameter	Threshold	Medical Condition
Heart Rate LOW	< 50 BPM	Bradycardia: dangerously slow heart rate
Heart Rate HIGH	> 120 BPM	Tachycardia: dangerously fast heart rate
SpO2 LOW	< 94%	Hypoxia: insufficient blood oxygen saturation
Temperature LOW	< 35.0 degrees C	Hypothermia: abnormally low body temperature
Temperature HIGH	> 38.0 degrees C	Fever: elevated body temperature

NOTE: Guard conditions (spo2 > 0 AND hr > 0) prevent false alerts during startup before the MAX30100 acquires a valid reading. Without these guards, every boot would trigger a false red LED and Firebase alert.

PART B - VIVA PREPARATION

Every question below is phrased exactly as your lecturer will phrase it. Answers are written to show engineering understanding, not memorisation. Read each answer out loud at least twice before your viva.

CHAPTER 7

WHY Questions - Hardware and Design Decisions

Q1: Why did you move from a 3-wheel (2WD + ball caster) to a 4-wheel drive system?

The original 2WD configuration with a rear ball caster was mechanically unstable during line following. The ball caster is a passive pivot point - when motors start or stop suddenly, the caster introduces lateral drift, causing the robot to veer off the line track.

4WD eliminates the caster entirely. All four wheels are driven simultaneously, distributing torque evenly. This gives uniform traction across the chassis, prevents skidding at motor start-up, and makes turning predictable: both left wheels reverse while both right wheels go forward, creating a clean zero-radius pivot turn with no unpredictable passive pivot.

The trade-off is increased current draw (approximately 800mA additional for two extra motors). Our LM2596 buck converter is rated 3A and our total load is approximately 1.1A, so this is well within capacity. The improved navigation accuracy justified the additional current consumption.

Q2: Why did you choose the ESP32 over the Arduino Uno as the main controller?

The Arduino Uno is an 8-bit microcontroller at 16MHz with 2KB SRAM and 32KB flash. It has no WiFi, no Bluetooth, and no built-in TLS/SSL capability. To connect to Firebase, it needs an ESP32 as a separate WiFi gateway, creating two controllers and a serial bridge.

The ESP32 is a 32-bit dual-core processor at 240MHz with 520KB SRAM, 4MB flash, built-in WiFi with TLS 1.2 support, hardware I2C, hardware SPI, and 34 GPIO pins. One ESP32 replaces both the Arduino and the ESP32E from the original design.

Specifically for our project: the ESP32 runs the MAX30100 sensor task on Core 0 while Firebase communication runs on Core 1 - true hardware parallelism that an Arduino simply cannot do. This eliminates the sensor-blocking problem that arises when Firebase SSL writes occupy the processor for hundreds of milliseconds.

Q3: Why did you use a buck converter instead of a linear voltage regulator or resistors?

A linear regulator (like a 7805) and a resistor both drop voltage by wasting excess energy as heat: Power wasted = (Vin minus Vout) multiplied by current. At our load of 1.1A, dropping from 8V to 5V wastes 3V x 1.1A = 3.3W continuously as heat. This drains the battery fast and heats the component.

The LM2596 is a switching buck converter at 150kHz. It achieves 75-80% efficiency by rapidly switching a transistor and storing energy in an inductor rather than burning the excess. At 1.1A load, it wastes only approximately 0.9W versus 3.3W for a linear regulator.

Over a 2-hour battery run: the linear regulator wastes 3.3W x 7,200s = 23,760 joules. The LM2596 wastes approximately 6,480 joules. That saved energy directly extends battery runtime. The LM2596 is also rated at 3A continuous output - far exceeding our 1.4A design current with comfortable margin.

Q4: Why did you remove the RFID scanner from the design?

The RC522 requires 5 SPI pins: MOSI, MISO, SCK, SS, and RST. In the single-ESP32 architecture, after allocating pins for DHT22 (1), MAX30100 I2C (2), 5x IR sensors (5), HC-SR04 (2), and L298N motor driver (6), we had insufficient SPI-capable GPIO pins remaining without conflicts. The ESP32's hardware SPI buses have default pins that overlap with pins already used for motor control.

The RC522 also operates strictly at 3.3V logic, but several remaining GPIO pins share voltage domains with 5V motor driver circuitry. Adding level shifters for 4 SPI lines would have required additional components and wiring complexity close to the demo deadline.

The replacement - an all-black IR sensor reading at a black tape cross-marker - is equally deterministic, requires no extra pins, has no voltage incompatibility, and is actually more reliable. RFID cards can fail to scan if the angle or distance is slightly off. The optical marker is detected reliably every time the robot reaches the position.

Q5: Why are resistors used on the HC-SR04 ECHO pin?

The HC-SR04 is a 5V device. Its ECHO pin outputs a 5V HIGH signal when echo is detected. The ESP32 GPIO pins have an absolute maximum input voltage of 3.6V. Connecting a 5V signal directly to an ESP32 GPIO will damage the internal protection diode over time or immediately cause permanent GPIO failure.

We use a resistor voltage divider: 1k ohm from ECHO to GPIO23, and 2k ohm from GPIO23 to GND. When ECHO is HIGH (5V): V at GPIO = 5 x 2,000 / (1,000 + 2,000) = 3.33V. This is safely within the ESP32's 3.3V tolerance.

The TRIG pin is an output from the ESP32 at 3.3V. The HC-SR04 TRIG threshold is approximately 2V, so 3.3V is sufficient to trigger it. No resistor or level shifter is needed on TRIG - only on ECHO which carries the 5V return signal.

Q6: Why are resistors used on the LED connections?

LEDs are diodes - without a current limiting resistor they will draw as much current as the circuit can supply. A typical red or green LED has a forward voltage of approximately 2.0V. Connected directly to a 3.3V GPIO: current = (3.3 - 2.0) / Rled. The LED's internal resistance Rled is very low (a few ohms), so current would be 100-300mA.

The ESP32 GPIO maximum safe output current is 12mA per pin. Exceeding this permanently damages the GPIO output driver. The LED itself may also burn out at high current.

Our 470 ohm resistor limits current to (3.3 - 2.0) / 470 = 2.8mA. This is well within the 12mA GPIO limit and produces clearly visible LED brightness. The formula is: R = (Vsupply - Vled) / Iled. For 10mA brightness: R = (3.3 - 2.0) / 0.010 = 130 ohms minimum. We use 470 ohms for extra safety margin.

Q7: Why are pull-up resistors needed on the I2C lines for the MAX30100?

I2C is an open-drain bus - devices can only pull the line LOW, they cannot actively drive it HIGH. The lines must be pulled HIGH by external resistors when no device is transmitting. Without pull-up resistors, the SDA and SCL lines float at an undefined voltage and I2C communication is completely unreliable.

We connect 4.7k ohm resistors from SDA to 3.3V and from SCL to 3.3V. The value matters: too large (above 10k ohm) and the pull-up is too weak, causing the line to rise too slowly and corrupting data at higher speeds. Too small (below 1k ohm) and the current drawn when any device pulls the line LOW is excessive. 4.7k ohm is the standard recommended value for I2C at 100kHz to 400kHz.

The resistors pull to 3.3V specifically because the MAX30100 is a 3.3V device. Pulling to 5V would exceed the MAX30100's input voltage maximum and damage it. Only one set of pull-ups per bus is needed regardless of how many I2C devices share the bus.

Q8: Why did you choose 5 IR sensors for line following instead of 2 or 3?

A 2-sensor array can only detect three states: on line, off left, off right. It cannot distinguish between a slight deviation and a large deviation. Motor correction is binary - full left turn or full right turn - causing the robot to oscillate side-to-side (zig-zag) while following the line.

A 5-sensor array allows proportional steering. Centre sensor S3 LOW = forward with no correction. S2 or S4 LOW (adjacent to centre) = slight turn. S1 or S5 LOW (outermost) = sharp turn. This graduated response produces smooth stable line following at higher speeds.

The fifth critical state - all five sensors LOW simultaneously - gives us the patient stop-point detection that replaces RFID. A 3-sensor array could not provide both proportional steering and reliable stop-point detection at the same time.

Q9: Why did you replace MLX90614 with DHT22?

The MLX90614 uses I2C at address 0x5A. The MAX30100 also uses I2C at address 0x57. Both share the same SDA/SCL bus. During testing, simultaneous read operations caused bus contention - the I2C bus would be held by one sensor during a slow read, causing the other to time out and produce failed reads.

The MLX90614 also requires precise targeting: 5-15cm distance, perpendicular angle, and emissivity calibration for human skin. On a moving robot that approaches each patient from a slightly different angle each time, maintaining this calibration is impossible.

The DHT22 uses a completely separate single-wire protocol on its own GPIO pin with zero I2C bus conflict. It adds humidity as an additional health metric, and its temperature accuracy of plus or minus 0.5 degrees C is sufficient for indicating fever. The trade-off is ambient air temperature rather than body surface temperature, but for a prototype demonstration this is acceptable.

Q10: Why did you simplify obstacle avoidance from dynamic reverse-and-turn to stop-and-wait?

The original reverse plus turn avoidance sequence caused the robot to lose the line track. After reversing and turning approximately 90 degrees, none of the 5 IR sensors were over the line - the robot would enter an undefined state and either spin or head in the wrong direction.

In a hospital corridor, obstacles are almost always temporary: a nurse walking past, a trolley being moved. The clinically appropriate response is to wait briefly, not attempt dynamic navigation around an unknown obstacle. A robot reversing unexpectedly in a hospital ward is also a patient safety hazard.

The stop-and-wait approach maintains the robot's exact position and heading on the line. When the obstacle clears, the robot simply resumes line following from the same position. It also logs the event to Firebase (/robotStatus/currentState = OBSTACLE), allowing nursing staff to know the robot needs assistance if it waits longer than expected.

CHAPTER 8

HOW Questions - Technical Deep Dives

Q11: How does the voltage divider on the ECHO pin work mathematically?

A voltage divider consists of two resistors in series between the source voltage and GND. The output voltage is taken from the junction between them.

Our divider: ECHO (5V) connects to 1k ohm, then to the junction node (connected to ESP32 GPIO23), then 2k ohm to GND. Using the voltage divider formula: Vout = Vin x R2 / (R1 + R2) = 5V x 2,000 / (1,000 + 2,000) = 5 x 0.667 = 3.33V.

This 3.33V sits safely within the ESP32 GPIO absolute maximum of 3.6V. The two resistors also limit short-circuit current: if the GPIO were to accidentally go HIGH while ECHO is HIGH, the maximum current is 5V / 3k ohm = 1.67mA, harmless for both the sensor and the GPIO.

Q12: How does the MAX30100 measure heart rate and SpO2 without touching inside the body?

The MAX30100 uses photoplethysmography (PPG). It has two LEDs - red at 660nm and infrared at 880nm - and a photodetector, all on the same side of the chip.

When placed on a fingertip, light passes through the tissue and is reflected back to the detector. Oxygenated haemoglobin (HbO2) absorbs infrared light strongly but reflects red light. Deoxygenated haemoglobin (Hb) does the opposite. By computing the ratio of red to IR absorption (the R-value), the sensor calculates SpO2 percentage.

Heart rate is derived from the pulsatile component of the signal: each heartbeat causes a small increase in blood volume in the fingertip, momentarily changing the absorption. The beat detection algorithm counts these pulses over time to calculate BPM. This is identical in principle to the pulse oximeter used in hospitals.

Q13: How does the LM2596 buck converter regulate voltage?

The LM2596 is a switching regulator. It works by rapidly turning a MOSFET transistor ON and OFF at 150kHz. When ON: current flows from input through the transistor and through an inductor to the output, storing energy in the inductor's magnetic field. When OFF: the inductor releases its stored energy to maintain output current through a freewheeling diode.

A feedback network (the blue trim potentiometer) compares the output voltage to an internal 1.23V reference. If the output is too high, the duty cycle (ON time percentage) decreases. If too low, duty cycle increases. This feedback loop maintains a stable output voltage regardless of input fluctuations or load changes.

The efficiency advantage: the switching transistor is either fully ON (low resistance, low heat) or fully OFF (no current, no heat). The inductor stores and releases energy without dissipating it. Only switching losses and inductor resistance contribute to inefficiency, giving 75-80% efficiency compared to 40-50% for a linear regulator.

Q14: How does line following work with 5 sensors?

Each IR sensor module has an infrared LED emitting 940nm light downward and a photodiode to detect the reflected light. White surfaces reflect most IR light back - photodiode triggers, output goes HIGH. Black surfaces absorb IR light - little reflected, output stays LOW.

The main loop reads all 5 sensor pins. Priority logic determines motor action: S3 LOW (centre on line) = both motors forward. S2 LOW (centre-left) = slight left turn. S4 LOW = slight right turn. S1 LOW (far left) = sharp left turn. S5 LOW = sharp right turn. All HIGH (no line) = stop.

The all-LOW condition (all five sensors over black tape simultaneously) triggers patient stop logic. This is the cross-marker at each patient bedside. The 5-sensor width ensures the cross-marker is reliably detected even if the robot approaches from a slight angle.

Q15: How does 4WD turning work without a steering servo?

4WD turning uses differential drive: the left and right motor pairs are driven in opposite directions. To turn left: right motors forward (IN3=HIGH, IN4=LOW), left motors backward (IN1=LOW, IN2=HIGH). The robot pivots around its own centre point - called a zero-radius turn or point turn.

For line following, we use partial differential: one side slows down rather than reversing, creating a gradual curve rather than a sharp pivot. This keeps the robot closer to the line during turns.

The L298N's ENA and ENB PWM pins control speed independently for each side. analogWrite(ENA, 150) on the left with analogWrite(ENB, 255) on the right creates a left curve. This PWM speed differential is the same steering principle used in tank tracks and industrial mobile robots.

Q16: How do you calculate LED resistor values?

The formula is: R = (Vsupply - Vled) / Iled

Where: Vsupply = 3.3V (ESP32 GPIO HIGH). Vled = forward voltage of the LED (red LED approximately 2.0V, green LED approximately 2.1V). Iled = desired current. We choose 10mA as a safe visible value within GPIO limits.

R = (3.3 - 2.0) / 0.010 = 1.3 / 0.010 = 130 ohm minimum. We use 470 ohm for extra safety margin, giving 2.8mA - clearly visible and far below the 12mA GPIO limit. Always round up to the next standard resistor value and verify calculated current is below the GPIO maximum rated output.

Q17: How does the robot know it has reached a patient bedside without RFID?

We place a black tape cross-marker (approximately 5cm x 5cm) at each patient stop position on the line track. The cross is wide enough to cover all 5 IR sensor positions simultaneously.

When the robot reaches the cross, all 5 sensors read LOW simultaneously (all detecting black surface). The main loop checks: if NOT digitalRead(S1) AND NOT digitalRead(S2) AND NOT digitalRead(S3) AND NOT digitalRead(S4) AND NOT digitalRead(S5) - this 5-way AND condition triggers the stop logic.

The robot calls stopMotor(), updates Firebase to MEASURING state, then delay(5000) to allow the MAX30100 to collect a stable reading. The 5-second pause is long enough for the MAX30100 beat detection algorithm to converge on accurate HR and SpO2 values.

Q18: How does the FreeRTOS dual-core setup work for the MAX30100?

The ESP32 has two CPU cores: Core 0 and Core 1. Normally Arduino's setup() and loop() both run on Core 1. FreeRTOS allows creating separate tasks pinned to specific cores.

We call xTaskCreatePinnedToCore(poxTask, 'pox', 4096, NULL, 1, NULL, 0). This creates a task named poxTask with 4096 bytes of stack, priority 1, pinned to Core 0. poxTask calls pox.update() every 1ms using vTaskDelay(1) and reads HR/SpO2 every second.

The main loop on Core 1 handles Firebase, motor control, and navigation without ever blocking pox.update(). The shared variables g_hr and g_spo2 are declared volatile (prevents compiler optimisation) and protected with portENTER_CRITICAL / portEXIT_CRITICAL with a portMUX_TYPE spinlock to prevent both cores accessing the variables simultaneously.

CHAPTER 9

What-If and Scenario Questions

Q19: What happens if one motor stops working during the demo?

In 4WD, each side has two motors wired in parallel. If one of the four motors fails open-circuit (wire breaks), the other motor on that side continues driving. The robot loses some torque on that side but can still navigate, though with a slight directional pull.

If an entire L298N channel fails, the robot will spin in circles. The line following algorithm would attempt to correct but without any drive on one side, it cannot move forward.

To mitigate this: we tested each motor channel independently before the final demonstration. Motor PWM is set to 200 out of 255 (78% speed) rather than full speed, which reduces motor heating and extends brush life. A spare L298N module is prepared.

Q20: What if the buck converter is adjusted to the wrong voltage before connecting the ESP32?

This is a critical safety issue. A buck converter outputting 12V instead of 5V would instantly and permanently destroy the ESP32's internal regulator, all connected 3.3V sensors, and any active GPIO pins.

The safe procedure: connect ONLY the input to the LM2596 with NO load connected. Set multimeter to DC 20V range. Probe the output terminals. Adjust the blue trim potentiometer until the display reads exactly 5.00V. Apply a drop of clear nail varnish to lock the potentiometer setting. Only then connect the load.

After adjustment, verify voltage again under full load to confirm the LM2596 maintains regulation. A quality LM2596 will hold within plus or minus 0.1V under load.

Q21: What if your lecturer asks why you did not use a servo motor for steering instead of differential drive?

Servo-based Ackermann steering (like a car) requires a separate steering servo, linkage rods, and front axle hardware. It also requires precise servo angle control mapped to turning radius - significantly more mechanical complexity.

Differential drive with 4WD achieves the same result purely through software: varying the PWM ratio between left and right motor pairs. No extra hardware, no mechanical linkages, and turning radius can be changed dynamically in code by adjusting the speed difference between sides.

Differential drive is also the standard for mobile robots in constrained environments - hospital corridors, warehouses, factory floors. The zero-radius turn capability is a specific advantage over Ackermann steering in tight spaces near patient beds.

Q22: What would you improve if you had more time?

Six specific improvements. First: add a PN532 RFID module (I2C variant instead of SPI) to avoid the SPI pin conflict - the PN532 operates over I2C, sharing the MAX30100 bus with a different address.

Second: replace DHT22 with a dedicated forehead/wrist thermometer module for true body temperature measurement - ambient temperature is clinically less useful than skin temperature.

Third: add a second HC-SR04 at the rear of the robot - if someone approaches from behind while the robot is at a patient station, the rear sensor would detect them and prevent a collision.

Fourth: implement proper Firebase Authentication with anonymous sign-in. Current test-mode rules allow anyone with the database URL to read and write patient data.

Fifth: add motor encoders on each wheel shaft to enable closed-loop speed control. Currently both sides run at identical PWM values, but if motor efficiencies differ slightly the robot drifts. Encoders allow the software to actively correct for this drift.

Sixth: add a small OLED display on the robot chassis showing real-time vitals. This eliminates the need to check the phone app during demos and makes the system self-contained.

Q23: Is this robot safe to use in a real hospital?

In its current form, no - and we say this honestly. This is an academic prototype demonstrating IoT and robotics concepts, not a certified medical device.

Specific safety gaps: The MAX30100 is a monitoring-grade sensor, not FDA/CE certified for clinical use. Its SpO2 readings require calibration against a certified reference device. The robot has no hardware watchdog - if the ESP32 crashes, motors may continue running until the battery depletes. There is no emergency stop button accessible from outside the robot chassis.

However, the fundamental architecture - IoT sensors, cloud aggregation, real-time alerting, remote control - is identical to commercially deployed hospital robots like Aethon TUG and Moxi. The engineering principles are sound. Making it clinically deployable would require certified medical sensors, redundant safety circuits, mechanical safety guards, and regulatory approval under the relevant medical device directive.

Q24: Prove the battery can last 2 hours. Walk through the calculation.

We calculated this systematically. Current draw: ESP32 WiFi active 170mA. MAX30100 25mA. DHT22 1.5mA. HC-SR04 15mA. 5x IR sensors 50mA. L298N logic 36mA. 4x motors running 800mA. LEDs 14mA. Total while moving: approximately 1,111mA.

The robot does not run motors continuously. It moves to each patient station (approximately 30 seconds), then stops for measurement (approximately 10 seconds). Stationary current drops to approximately 311mA (no motors). At 75% moving / 25% stationary duty cycle: average current = (0.75 x 1,111) + (0.25 x 311) = 833 + 78 = 911mA.

Runtime = 2,500mAh / 911mA = 2.74 hours. With a 30% safety margin for battery ageing and temperature effects: 2.74 x 0.7 = 1.92 hours. This marginally meets the 2-hour requirement. Using 3,000mAh cells gives 3,000 / 911 x 0.7 = 2.3 hours, comfortably above the requirement.

CHAPTER 10

Quick-Fire Q&A; Reference Table

Memorise these cold. Your lecturer may fire them one after another without pause.

Question	Answer
Why 4WD over 2WD?	Stability and traction. 2WD ball caster drifts laterally. 4WD distributes torque evenly and turns predictably with zero radius.
Why ESP32 over Arduino?	Built-in WiFi, dual-core 240MHz, 34 GPIO pins, hardware I2C/SPI, TLS 1.2 support. Arduino has none of these.
Why LM2596 over a 7805 linear regulator?	75-80% efficiency vs ~40%. At 1.1A load, the 7805 wastes 3.3W as heat continuously; LM2596 wastes only ~0.9W.
What voltage does HC-SR04 ECHO output?	5V HIGH - must be divided to 3.3V before connecting to ESP32 GPIO to prevent damage.
Voltage divider formula?	Vout = Vin x R2 / (R1 + R2). For 5V to 3.3V: use 1k ohm and 2k ohm.
Why 4.7k ohm pull-up resistors on I2C?	I2C is open-drain: devices can only pull LOW. External resistors pull lines HIGH. Without them I2C will not work at all.
Why pull I2C to 3.3V and not 5V?	MAX30100 is a 3.3V device. 5V on its SDA/SCL pins exceeds absolute maximum input voltage and damages it.
What is the MAX30100 I2C address?	0x57 (fixed, cannot be changed by software)
LED resistor formula?	R = (Vsupply - Vled) / Iled. For 3.3V GPIO red LED: (3.3-2.0)/0.010 = 130 ohm minimum. Use 470 ohm for safety.
Why 470 ohm for LEDs specifically?	Gives 2.8mA: visible and within ESP32 GPIO 12mA maximum limit. Prevents GPIO output driver damage.
Why not use Arduino 3.3V pin for sensors?	Arduino LP2985 regulator provides only 50-150mA. ESP32 alone draws 350mA peak during WiFi transmission - causes brownout resets.
Why not use L298N's built-in 5V for logic?	L298N internal 78M05 regulator provides ~500mA but sags under heavy motor load. Use external buck converter instead.
What is back-EMF and why does it matter?	When a motor stops suddenly, spinning coils generate a reverse voltage spike that can destroy the L298N and ESP32 GPIO pins. Freewheeling diodes absorb these spikes.
Why decoupling capacitors on sensors?	Motor brushes generate high-frequency electrical noise that travels through power lines and corrupts I2C sensor readings. 100nF caps filter this noise at the source.
What is the HC-SR04 trigger pulse duration?	10 microseconds HIGH on TRIG pin. Sensor then emits 8 ultrasonic bursts at 40kHz and measures echo return time.

What does analogWrite do on ESP32?	Generates a PWM signal. Value 0-255 maps to 0-100% duty cycle. Used for motor speed control on ENA/ENB pins of L298N.
What does vTaskDelay(1) do?	Yields the FreeRTOS task for 1ms tick. Prevents poxTask from monopolising Core 0 and allows RTOS scheduler to run other tasks.
All 5 IR sensors LOW simultaneously means?	Patient stop point detected: black cross-tape marker under all 5 sensors. Robot stops for 5-second measurement pause.
Why 4WD turns in place?	Left motors run backward, right motors run forward simultaneously. Robot pivots around its own centre - zero turning radius, no drifting.
Why must battery negative connect to all component grounds?	Without common ground, voltage measurements are relative to different reference points: sensors read wrong values, I2C fails, serial produces garbage.
What is the LM2596 switching frequency?	150kHz. This high frequency allows a smaller inductor and output capacitor compared to older switching regulators.
Why NTP sync before Firebase.begin()?	Firebase uses HTTPS with TLS certificates that have notBefore validity dates. ESP32 starts at Unix timestamp 0 (year 1970) on cold boot - before any valid certificate. Firebase rejects the SSL connection without NTP sync.
What does portENTER_CRITICAL do?	Disables interrupts and acquires a spinlock on the specified mutex, preventing any other task or core from accessing the protected code block simultaneously.
What is the patient stop-point detection condition?	All 5 IR sensors reading LOW simultaneously: NOT(S1) AND NOT(S2) AND NOT(S3) AND NOT(S4) AND NOT(S5) = true.

END OF DOCUMENT - Hospital Health Monitoring Robot Robotics Implementation Report
HNDCSAI-261F/P . NIBM School of Computing and Engineering